

QUERYHOLIC

STEP 2 — SCREEN / ROUTE / COMPONENT INVENTORY

Frontend Implementation Planning Document

Objective

Convert the completed high-fidelity Stitch UI into a precise frontend implementation map. Every screen, route, shared component, state, role boundary, and dependency must be identified before coding starts.

Critical rule

The existing UI/design system is the source of truth. Do not redesign the visual language during inventory or implementation. Reuse the established typography, colors, spacing, component patterns, navigation patterns, cards, forms, tables, states and responsive behavior.

1. Step 2 Deliverables

Deliverable	What must be produced
Screen Inventory	Complete list of every distinct screen/view in the project.
Route Map	Canonical route for every screen, grouped by product role.
Component Inventory	Reusable global, role-level and feature-level components.
State Inventory	Loading, empty, error, success, disabled, pending, rejected and edge states.
Interaction Map	Buttons, forms, filters, tabs, modals, drawers, navigation and primary actions.
Data Contract Map	What data each screen reads, writes, filters or displays.
Asset Inventory	Images, icons, logos, illustrations, fonts and other static assets.
Responsive Map	Desktop/tablet/mobile behavior for every major screen.
Implementation Gaps	Missing screens, duplicate patterns, inconsistent components and technical blockers.

2. Canonical Route Structure

Area	Route namespace	Purpose
Public / Marketing	/	Landing, discovery, company and lead-generation pages.
Customer	/customer/*	Customer marketplace, booking, tracking, account and support.
Provider	/provider/*	Provider onboarding, dashboard, catalog, orders, operations and earnings.
Delivery Partner	/delivery/*	Partner onboarding, jobs, pickup, delivery and earnings.
Admin / Operations	/admin/*	Platform monitoring, management, KYC, disputes, refunds, settlements and analytics.
Auth	/auth/*	Login, signup, verification, password recovery and session flows.

3. Screen Inventory — Documented Product Scope

The following inventory uses the Queryholic phase structure currently established in the project context. Where exact Phase 1–6 screen names are not available in the current source context, they are intentionally not invented.

Phase	Area	Known scope / screen groups
7	Customer Experience	Customer marketplace, discovery, service/provider selection, booking, payment, tracking, completion, review and account/support flows.
8A–8J	Provider	Onboarding, registration, business profile, KYC, verification, dashboard, catalog, pricing, availability, service areas, capacity, booking/order management, processing, workshop operations, pickup/delivery handoff, earnings, transactions, payouts and settlements.
9	Admin / Operations	Admin dashboard, order management, provider management, customer management, disputes/complaints, refunds, payment/settlement management, delivery/logistics monitoring, KYC review and platform analytics.
10	Delivery Partner	Onboarding, dashboard, available jobs, job details/accept, pickup workflow, pickup confirmation, active delivery/navigation, delivery confirmation, failed delivery/reschedule and partner earnings; extended job states may exist in the generated UI.
11	Disputes / Refunds	Integrated into Admin/Ops rather than treated as a separate product surface; dispute evidence, resolution and refund workflows.
12	Marketing / Platform	Marketing/landing surfaces, lead capture and platform-facing content/capabilities established in the project.

4. Role-by-Role Inventory Checklist

Customer

✓ Authentication and account states
✓ Home / discovery / category browsing
✓ Service detail and provider comparison
✓ Booking configuration and address selection
✓ Checkout and payment states
✓ Booking/order tracking
✓ Order detail and lifecycle history
✓ Review/rating
✓ Favorites, profile, addresses and preferences
✓ Support, complaints and refund/dispute entry points

✓ Notifications and empty/error states
--

Provider

✓ Registration and onboarding
✓ Business profile
✓ KYC/document submission and verification states
✓ Dashboard and operational KPIs
✓ Service catalog and variants
✓ Pricing and add-ons
✓ Working hours / availability
✓ Service areas and capacity
✓ Booking/order queue
✓ Processing/workshop stages
✓ Quality check and handoff
✓ Earnings, transactions, payouts and settlements
✓ Profile/settings/notifications

Delivery Partner

✓ Onboarding and verification
✓ Partner dashboard
✓ Available jobs
✓ Job details and accept/reject
✓ Pickup workflow
✓ Pickup confirmation and proof
✓ Active delivery/navigation
✓ Customer handoff and delivery proof
✓ Failed delivery / reschedule
✓ Job history
✓ Earnings and payout history
✓ Availability/profile/settings

Admin / Operations

✓ Dashboard and live operational overview
✓ Orders and lifecycle management
✓ Provider management
✓ Customer management

✓ Delivery monitoring
✓ KYC verification review
✓ Dispute/complaint management
✓ Refund management
✓ Payment and settlement management
✓ Analytics/reporting
✓ Support operations
✓ Audit/log activity

5. Component Inventory Strategy

Do not create a unique component for every screen. Identify repeated UI patterns and turn them into shared components.

Layer	Examples	Rule
Design Tokens	Colors, type scale, spacing, radius, shadows, breakpoints	One source of truth; match existing Stitch visual language.
Global UI	Button, Input, Select, Checkbox, Badge, Avatar, Tooltip, Modal, Drawer	Reusable across all roles.
Navigation	Header, Sidebar, Bottom Nav, Breadcrumbs, Tabs	Role-aware but visually consistent.
Data Display	Card, Table, Stat, Timeline, List, Status badge	Prefer shared primitives with variants.
Forms	Address form, KYC upload, service form, profile form	Schema-driven validation where possible.
Marketplace	Service card, Provider card, Price breakdown, Booking summary	Customer-focused reusable components.
Operations	Order row, Queue item, Job card, KPI card, Activity feed	Provider/Admin/Delivery variants should share primitives.
Feedback	Toast, Alert, Empty state, Error state, Skeleton	Every async screen needs explicit states.
Media	Image uploader, Gallery, Proof viewer	Private/sensitive files require controlled access.

6. State Inventory — Mandatory for Every Async Screen

State	Expected behavior
Loading	Skeleton/spinner matching the established visual language; prevent confusing layout shifts.
Empty	Explain why there is no data and provide the relevant next action.
Error	Human-readable error + retry/recovery path; never expose raw stack traces.
Success	Clear confirmation and appropriate next step.
Disabled	Explain why an action is unavailable where useful.
Pending	Show current progress/status without implying completion.
Rejected	Explain rejection reason and remediation path where applicable.
Offline / network	Graceful recovery and retry behavior for network-dependent flows.

7. Screen → Data Mapping

Screen group	Primary data
Customer Home / Discovery	service_categories, services, providers, provider_services, reviews, offers
Service / Provider Detail	services, service_variants, provider_services, provider_service_pricing, reviews, service_media
Booking / Checkout	customer_addresses, bookings, booking_items, booking_item_addons, coupons, payments
Customer Order Tracking	bookings, booking_status_history, service_jobs, delivery_jobs, pickups, deliveries, handoffs, delivery_proofs
Provider Dashboard	providers, bookings, service_jobs, provider_capacity, provider_earnings
Provider Catalog	services, service_variants, service_addons, provider_services, provider_service_pricing
Provider KYC	provider_documents, provider_document_reviews, provider_bank_accounts
Provider Earnings	provider_earnings, payouts, payout_items, settlements, payment_transactions
Delivery Dashboard	delivery_partners, delivery_jobs, pickups, deliveries, delivery_attempts, delivery_issues, delivery_earnings
Admin Orders	bookings, booking_items, booking_status_history, service_jobs, delivery_jobs, payments
Admin KYC	providers, provider_documents, provider_document_reviews, delivery_partner_documents
Admin Disputes / Refunds	disputes, dispute_evidence, dispute_resolutions, refunds, refund_transactions
Admin Analytics	analytics_events, daily_metrics, metric_snapshots
Marketing / Leads	provider_leads, delivery_partner_leads, business_leads, contact_inquiries, campaigns, offers, coupons

8. Asset & Design-System Audit

- Extract every image, icon, logo, illustration and font from the generated UI.
- Deduplicate visually identical assets and preserve the highest-quality source.
- Record asset purpose, source path, dimensions and whether it is public or sensitive.
- Map every repeated visual pattern to a reusable component instead of copying markup.
- Lock typography and spacing values before implementation.
- Do not introduce a new color palette, typography system, icon style or component language.

9. Responsive Implementation Matrix

Viewport	Implementation requirement
Desktop	Preserve the intended multi-column layouts, navigation and information density from Stitch.
Tablet	Collapse secondary columns and adapt navigation without changing the visual language.
Mobile	Use the established mobile patterns; prioritize primary actions and convert tables/grids to appropriate stacked/list representations.
Accessibility	Keyboard focus, labels, semantic structure, readable contrast and touch target sizing must be implemented without visual redesign.

10. Antigravity Execution Checklist

Use this document as the Step 2 implementation-planning contract. The first task is inventory, not redesign.

- 01 Read the entire exported UI project and identify the actual framework/build setup.
- 02 Enumerate every route and every distinct screen/view.
- 03 Group routes into Auth, Customer, Provider, Delivery, Admin/Ops and Public/Marketing.
- 04 Identify duplicated screens that are only state/variant differences.
- 05 Inventory every reusable component and create a component dependency map.
- 06 Inventory loading, empty, error, success, disabled, pending and rejected states.
- 07 Inventory every asset and map it to the screen/component where it is used.
- 08 Create the screen → component → data dependency map.
- 09 Create the route protection/RBAC map.
- 10 Identify missing screens or broken flows without changing the design.
- 11 Identify components that should be shared rather than duplicated.
- 12 Produce an implementation backlog ordered by foundation → shared components → role flows → integrations.
- 13 Do not change the established visual language during this audit.

11. Required Audit Output

Output	Format
Complete route list	Table: route, role, screen, auth requirement, status
Complete screen list	Table: screen, route, phase, primary action, dependencies
Component inventory	Table: component, location, variants, reuse candidates
State matrix	Table: screen, loading, empty, error, success, disabled, pending
Asset inventory	Table: asset, path, type, usage, sensitivity
Data dependency map	Table: screen → API/data source → fields
RBAC map	Role → allowed route/action/data
Responsive map	Screen → desktop/tablet/mobile behavior
Gap report	P0/P1/P2/P3 with exact reason and recommended action
Implementation backlog	Ordered task list with dependencies

12. Definition of Done

Step 2 is complete only when every actual route/screen in the exported project has been inventoried, every reusable component opportunity is documented, all async/edge states are identified, assets are mapped, role

access is known, screen-to-data dependencies are mapped, responsive behavior is recorded, and the remaining frontend implementation work is converted into an ordered backlog.

Important: This is an inventory/planning document. It must not authorize a visual redesign. The existing Stitch UI remains the visual source of truth.